

# The Code Trust Shift

From Human Authorship  
to Verifiable Evidence



Position Paper Synthesis / Autonomous  
Intelligence & Enterprise Risk

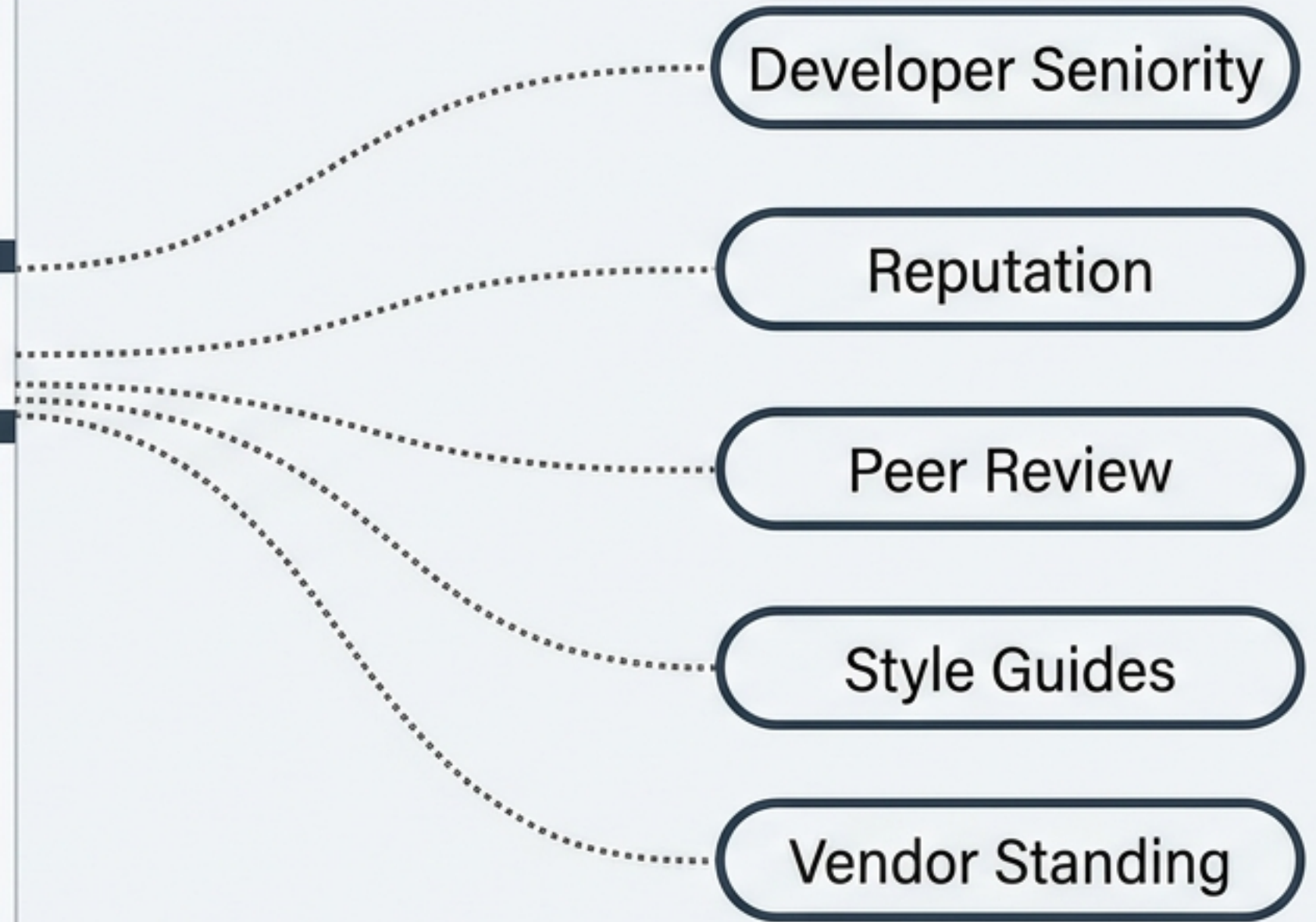


# The Inherited Assumption



For computing history, human programming was an immutable constraint. Software authorship was unavoidably a human bottleneck.

# The Proxies for Trust



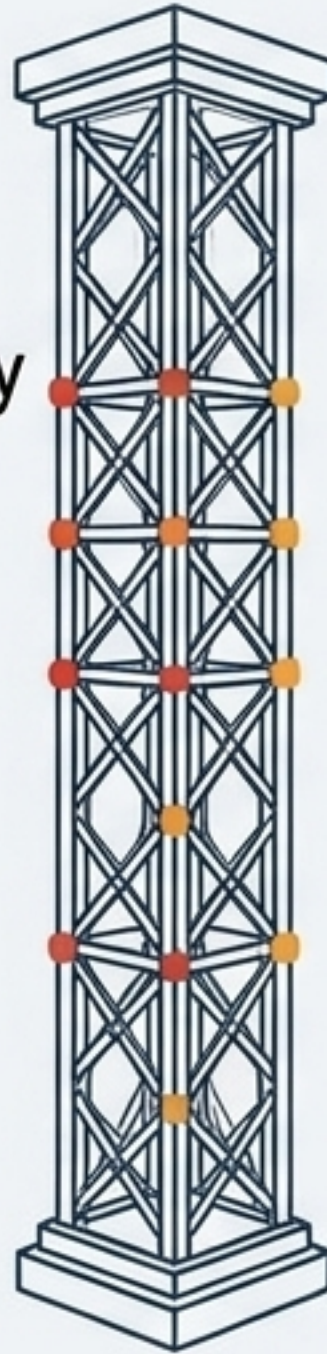
**Key Takeaway:** These proxies have value, but none formally demonstrates that an executable system will behave correctly under all critical conditions.



# The Hidden Vulnerabilities of Human-Centric Coding

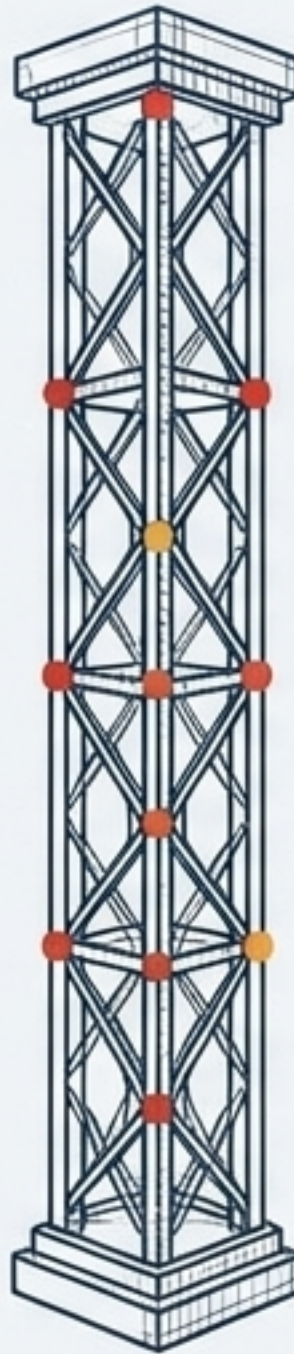
## Cognitive Limits

- Cognitive variability & fatigue
- Subjective implementation choices
- Effort-based incentives (rewarding hours over resilience)



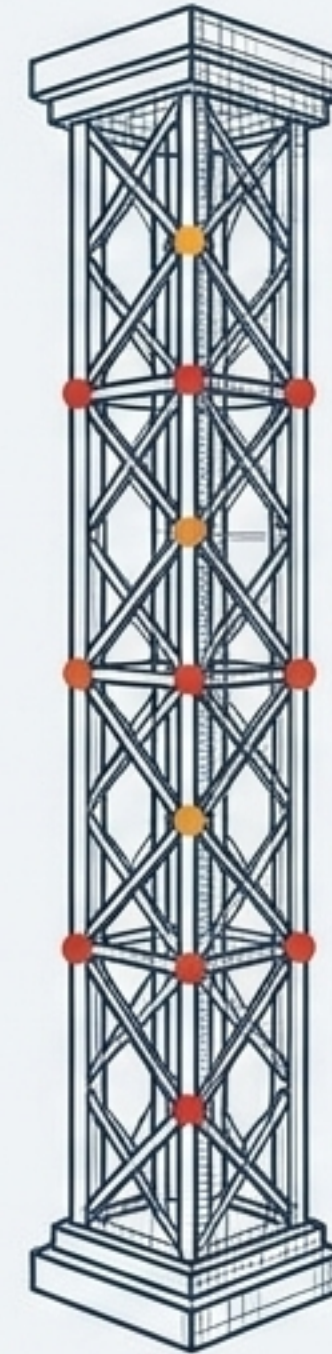
## Structural Gaps

- Interpretation loss from purpose to code
- Undocumented tacit knowledge & key-person dependency
- Human review as a weak substitute for evidence



## Systemic Vulnerability

- Inconsistent security discipline (e.g., CISA/FBI 2024 SQLi alerts)
- Broad insider credential exposure
- Scarcity-driven remediation delays
- Scaling individual decisions into systemic risk

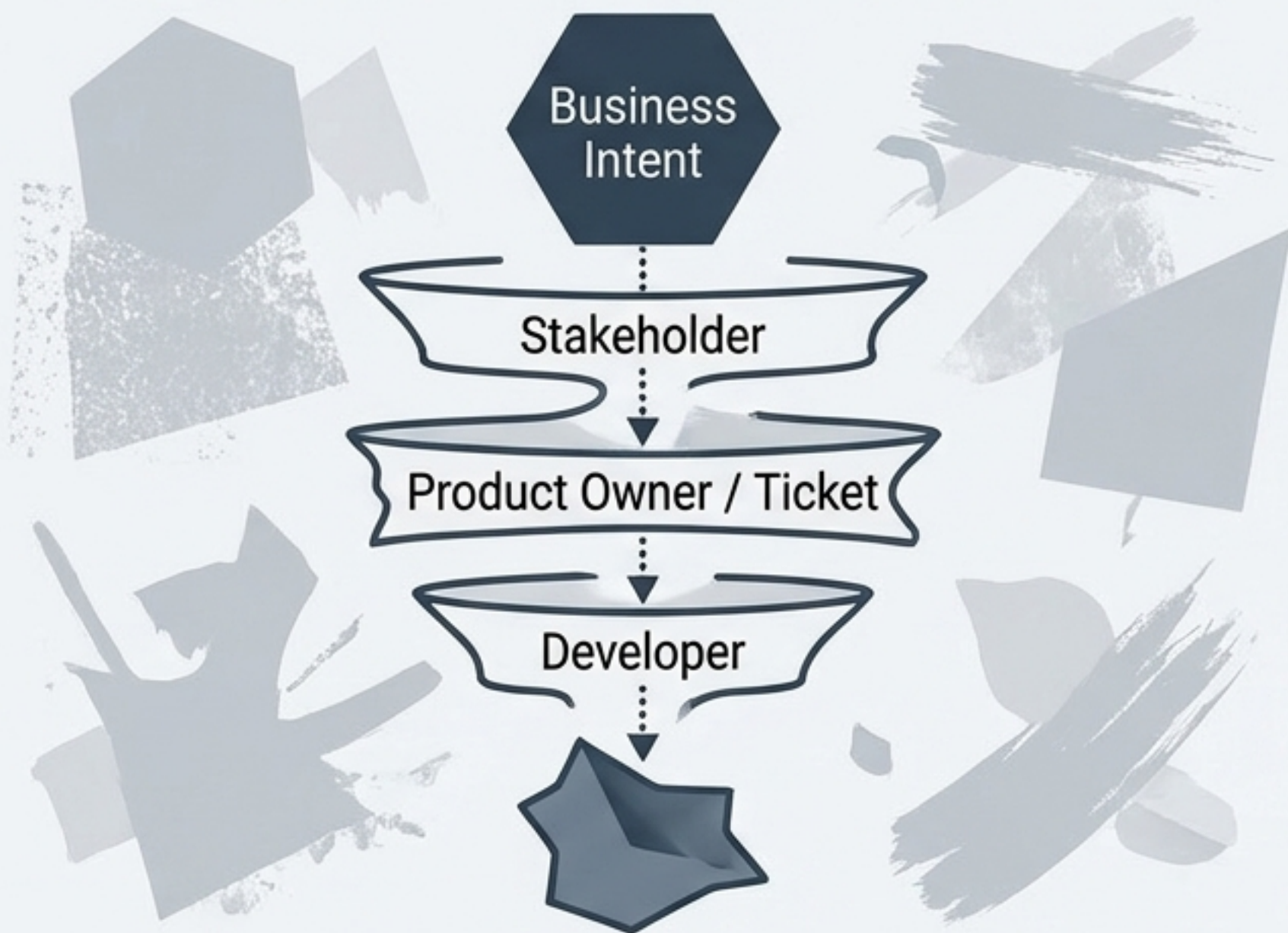


**Key Takeaway:** Human-centric coding creates vulnerabilities that are structural, systemic, and difficult to manage at scale.



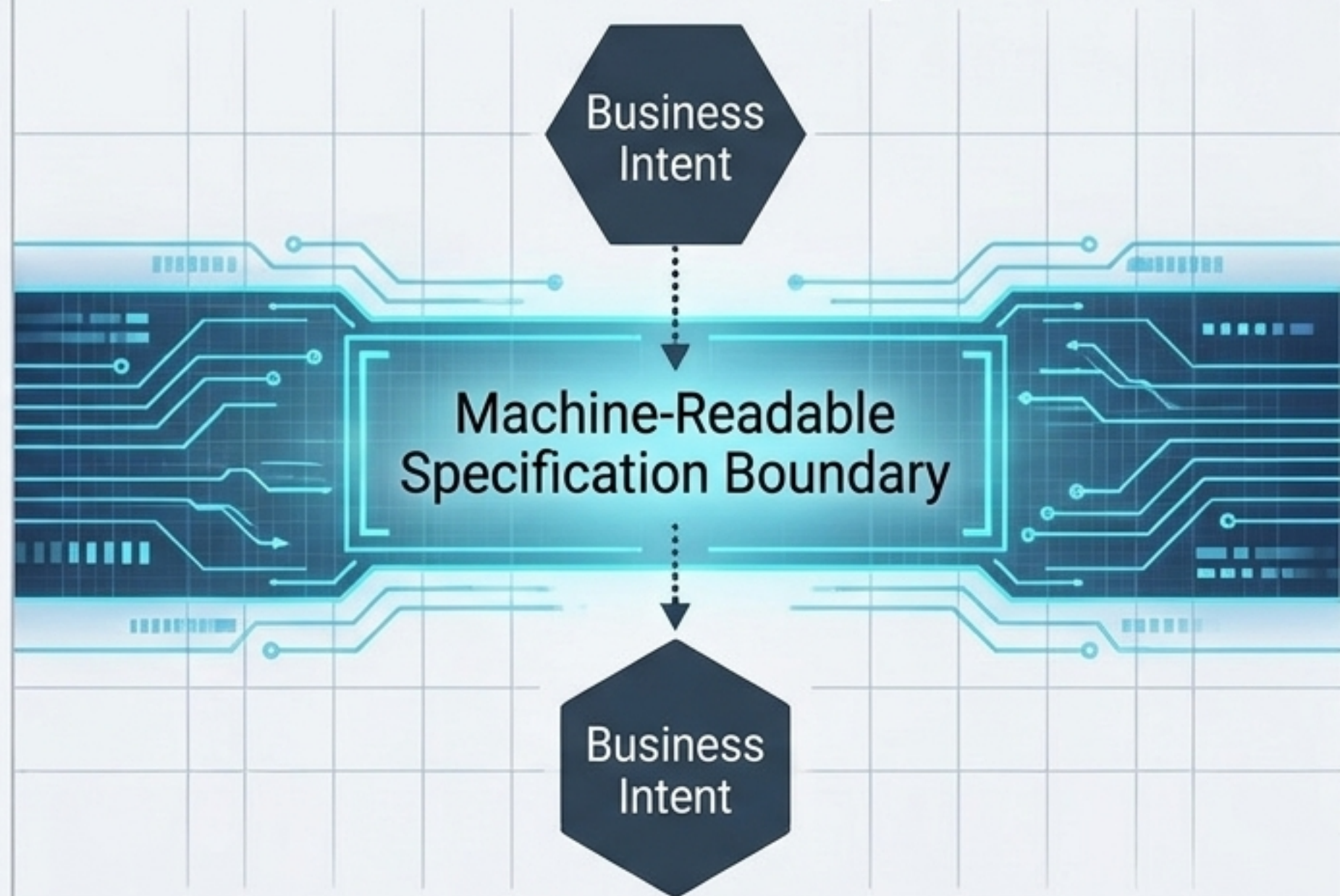
# The Anatomy of Interpretation Loss

## Legacy Model: Human Translation



Intent discarded or assumptions introduced at every handoff.

## New Model: Persistent Specification



Assumptions made visible and testable regardless of author.

**Warning: The eventual program may perfectly implement a misunderstanding.**



# The Illusion of AI Safety

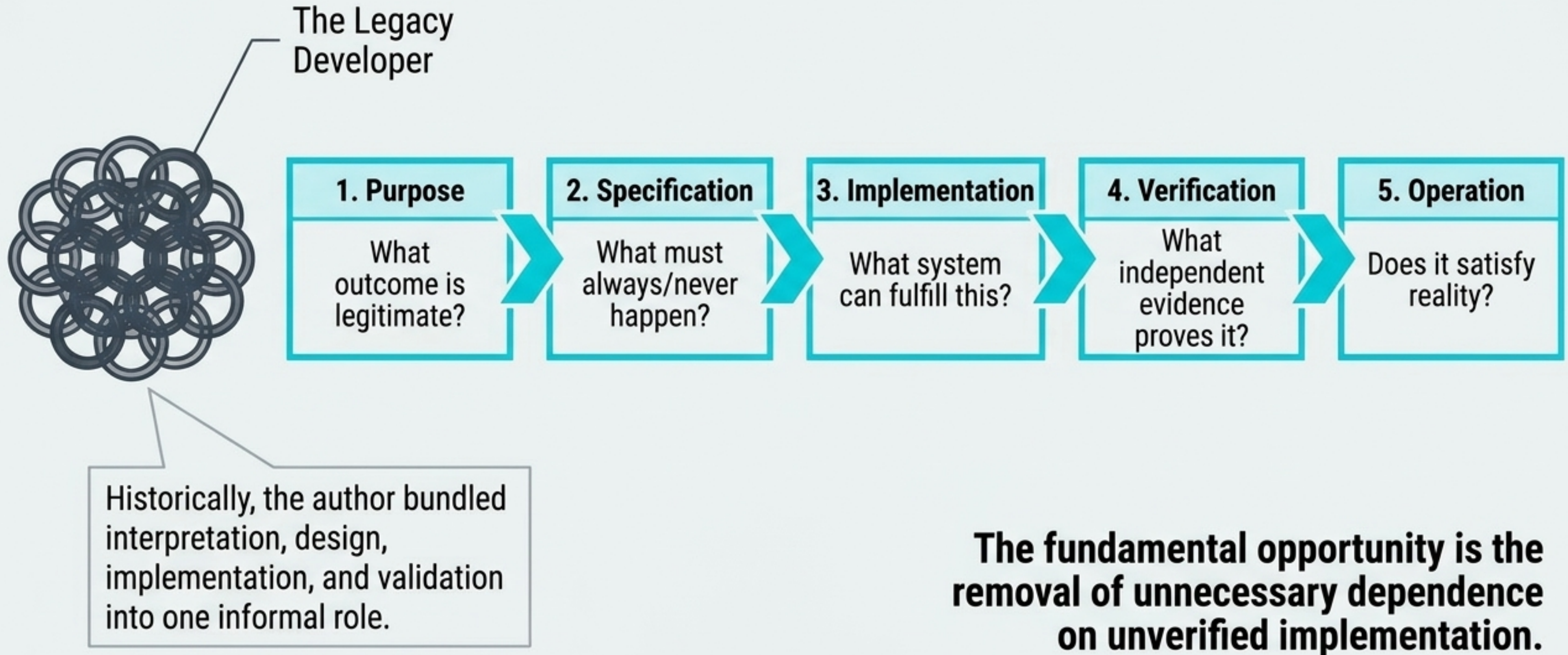
Both human-only and AI-only coding are inadequate assurance models.

| Human-Centric Vulnerabilities                                                                                            | Machine-Centric Vulnerabilities                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- Cognitive fatigue &amp; variability</li><li>- Tacit knowledge hoarding</li></ul> | <ul style="list-style-type: none"><li>- OWASP Agentic Risks: Goal hijacking, tool misuse, identity abuse, unexpected code execution.</li></ul>                                      |
| <ul style="list-style-type: none"><li>- Scarcity-driven patch delays</li></ul>                                           | <ul style="list-style-type: none"><li>- Performance Reality 1: 45% of generated samples failed security testing in 2025 Veracode evaluation.</li></ul>                              |
| <ul style="list-style-type: none"><li>- Subjective biases &amp; preferred-framework drift</li></ul>                      | <ul style="list-style-type: none"><li>- Performance Reality 2: METR 2025 study found experienced open-source devs working in mature repositories took 19% longer with AI.</li></ul> |

**Conclusion:** Google DORA 2025 characterizes AI as an amplifier of organizational strengths and weaknesses, not a standalone solution.



# The Bundled to Unbundled Transformation





# Authorship $\neq$ Assurance.

---

**The Conventional Framing asks:**  
Can we trust a machine to write  
code a human would otherwise  
write?

**The Engineering Framing asks:**  
**Why should we trust executable  
code merely because a human  
wrote it?**

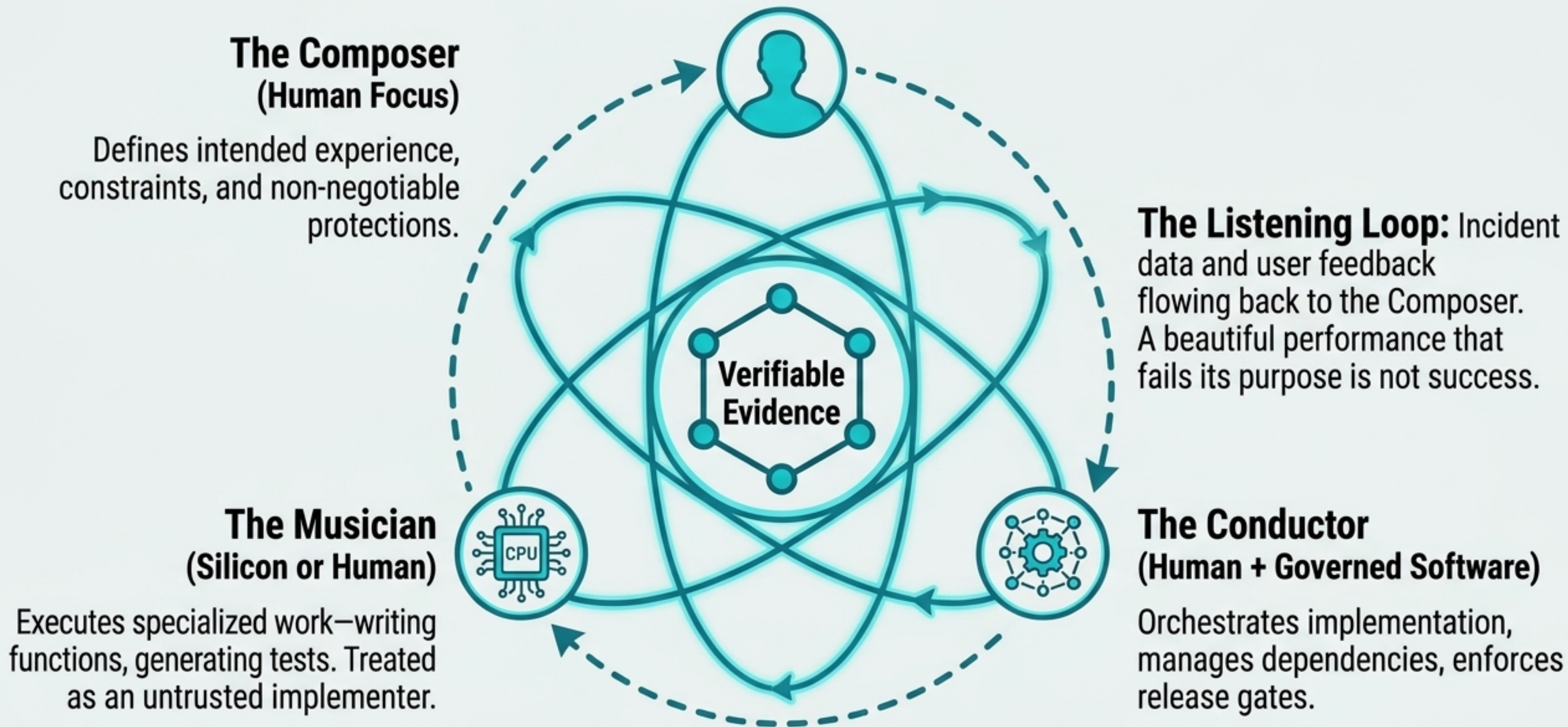


# The Shifting Architecture of Trust

|                       | Legacy Model (Trust the Author)                           | Modern Model (Trust the Evidence)                                  |
|-----------------------|-----------------------------------------------------------|--------------------------------------------------------------------|
| Unit of Trust         | The Person who wrote it.                                  | The <b>Evidence</b> that proves it.                                |
| Verification Standard | Pull Request / Peer Review.                               | Machine-readable <b>constraints</b> & adversarial <b>testing</b> . |
| Failure Mode          | Unstated assumptions & tacit knowledge.                   | Explicit, <b>bounded</b> failures.                                 |
| Scale Risk            | Individual judgment propagating to thousands of services. | <b>Containment</b> boundaries & independent <b>checks</b> .        |



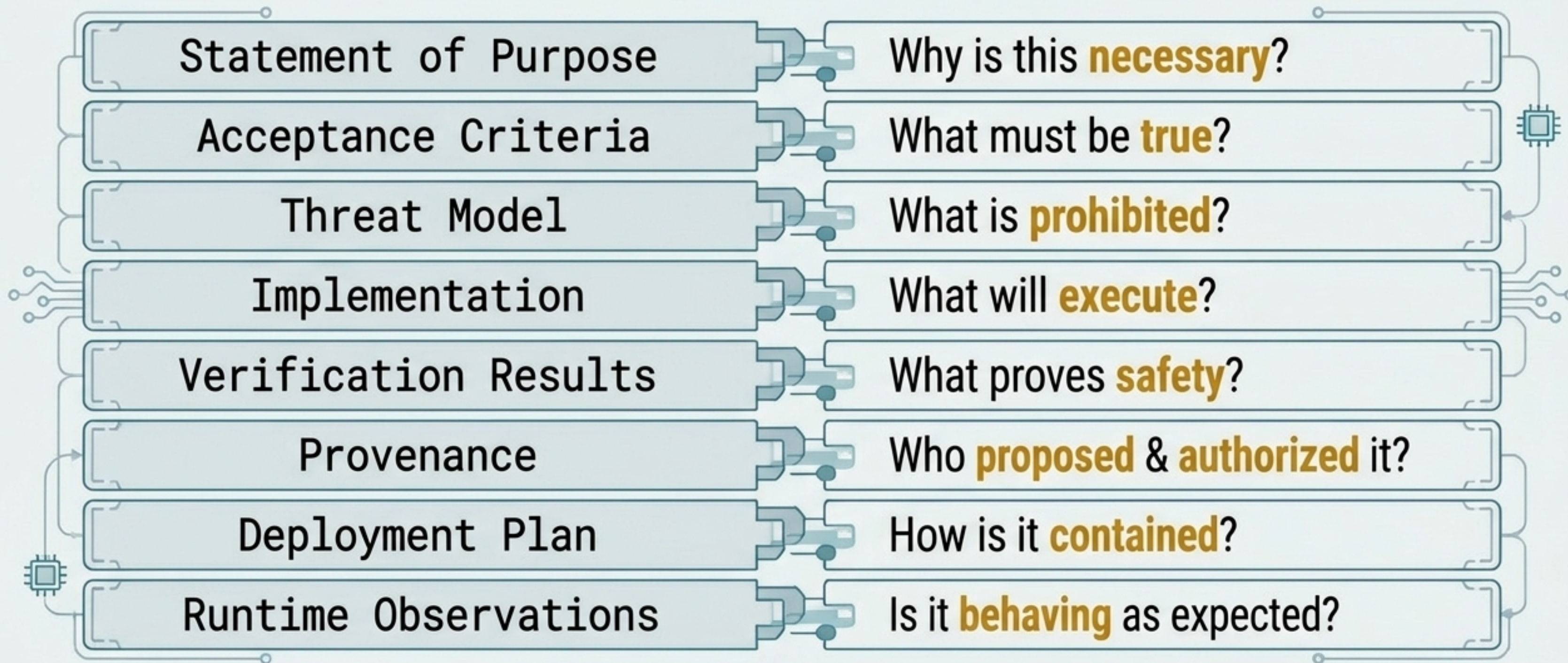
# The New Engineering Triad





# The Inspectable Evidence Package

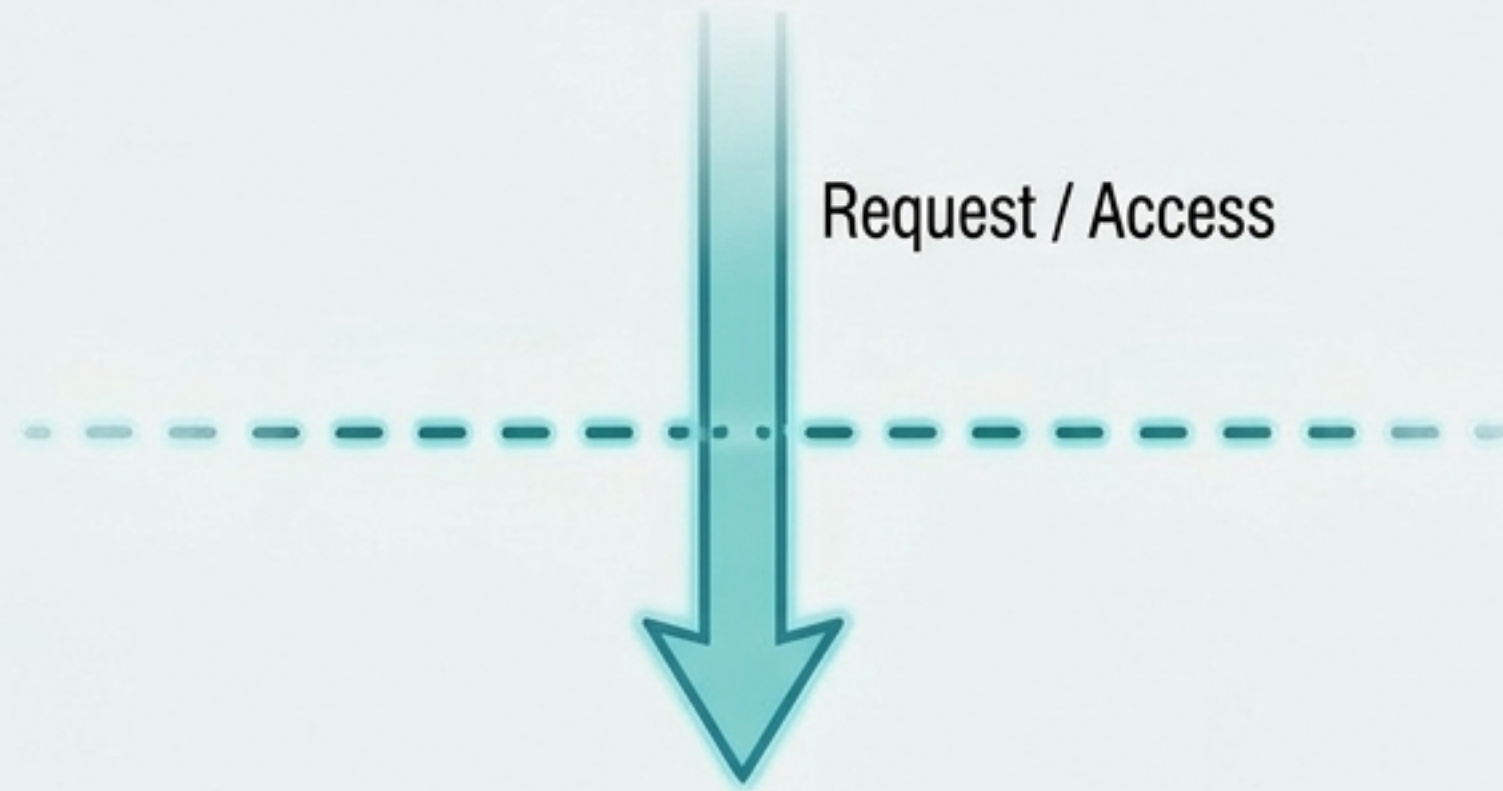
Bounded confidence, not impossible promises.





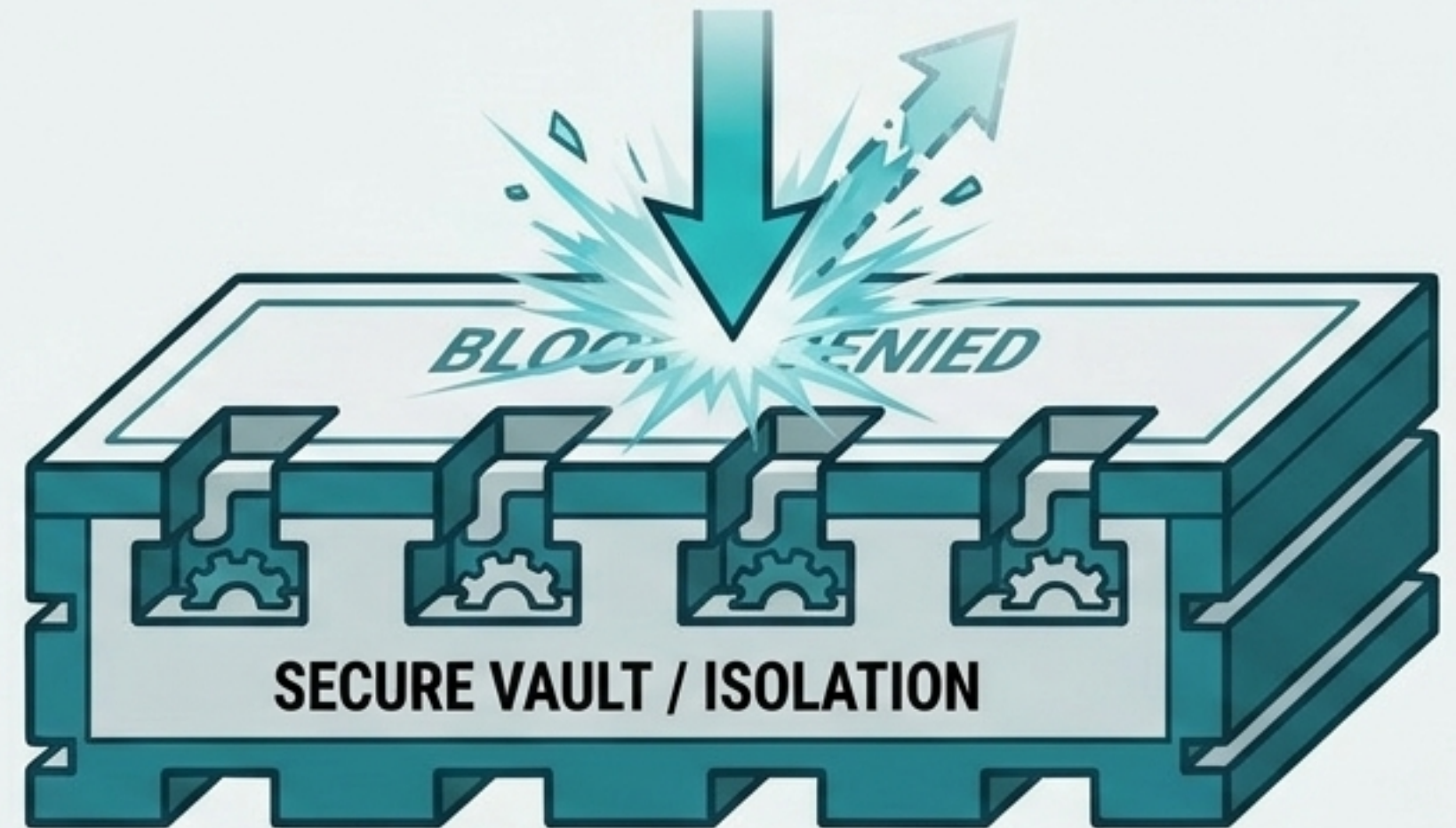
# Applied Evidence: Multi-Tenant Isolation

## Legacy: Weak Requirement



Users can view their organization's records.  
(Bypassable by human or AI error).

## Modern: Strong Specification



- Identity established by server, not client fields. Integration tests.
- Cross-tenant authority explicitly approved & logged exposing content.
- Default behavior = Denial.
- asynchronous paths.

Neither AI nor human is permitted to bypass the invariants. Evidence is generated independent of the implementer's confidence.



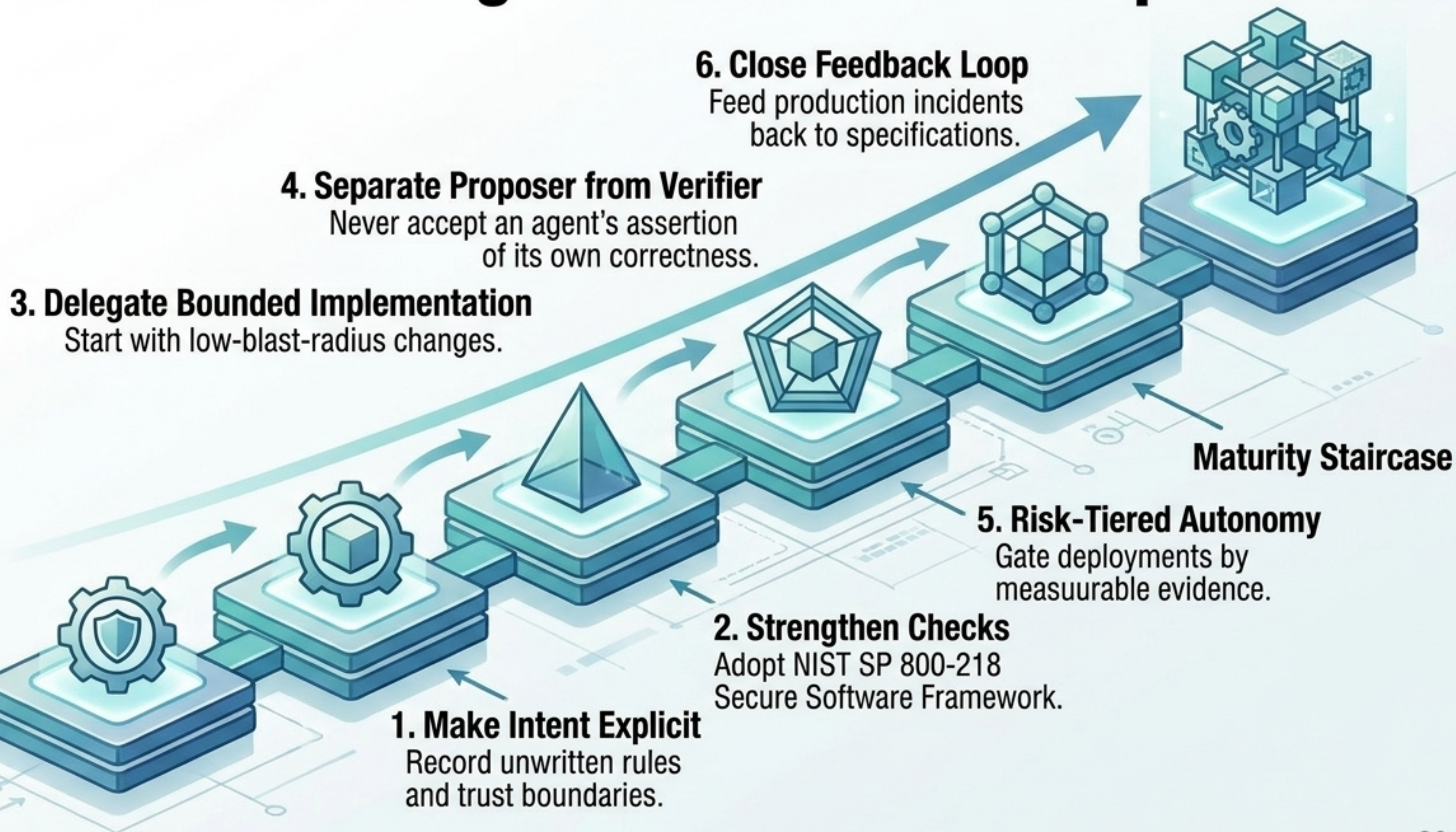
# Risk-Tiered Autonomy Diagnostic

|   | Risk Class    | Illustrative Work                                                 | Appropriate Default                                                             |
|---|---------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 1 | Low Risk      | Formatting, test scaffolding, bounded documentation.              | Automated generation with routine checks.                                       |
| 2 | Moderate Risk | Isolated features, dependency updates.                            | Agent proposal, automated verification, human review.                           |
| 3 | High Risk     | Authentication, multi-tenant isolation, PII handling.             | Strong specs, independent security checks, explicit accountable human approval. |
| 4 | Critical Risk | Reversible operations, privileged infrastructure, safety systems. | Restricted/no autonomous release; specialized assurance.                        |

*Note: Autonomy is earned by evidence, not granted by impressive AI demos.*



# The 6-Stage Transition Roadmap





# Governing by Outcome, Not Effort

Do not equate faster code production with better engineering.

## Speed & Efficiency Metrics



- 🕒 • Time to verified fix ☐
- 📊 • Recovery performance ☐
- 🔄 • Review and rework effort ☐

## Quality & Resilience Metrics



- Escaped defects (severity-weighted) ☐ ✓
- Security quality (verified vulnerabilities per change)
- Change failure rate ☐ ✓
- Specification traceability ☐ ✓

**Strategic Warning:** Do not measure human development rigorously while accepting AI output on faith—or vice versa. Continuously measure concentration risk (dependence on a single provider/model).



# Enterprise Governance Implications



## Human Capital

Hiring shifts from “writing code” to systems thinking, threat modeling, formal reasoning, and evaluating machine-produced work. Implementation ceases to be the default bottleneck.



## Procurement

Move beyond vendor claims of “experienced developers” or “newest AI model.” Demand inspectable evidence of provenance, security properties, testing, and incident response.



## Compliance

A signed commit does not establish safety. A legal entity and responsible people must dictate acceptable risk. Legal accountability cannot be delegated to a mathematical model.



## From Writing Code to Governing Behavior

**Humans—and the institutions  
they create—must remain  
remain accountable  
for what software does.**

The guiding question for the enterprise is no longer, “Who wrote this code?”  
***The question is, “What evidence justifies trusting this system to act?”***